

# Fenêtres SQL

## 12 cas corrigés



LAG, LEAD, RANK, running total : le cas, la requête, le piège partition/tri.

Une fonction fenêtre ne casse jamais la requête quand la partition ou le cadre sont faux : elle change juste le chiffre, en silence. J'ai vu un running total doubler une journée entière de CA en recette, à cause d'un cadre par défaut que personne n'avait vérifié. Douze cas, la requête, le piège nommé, la correction.

### Numéroter et classer.

*Ici, le rang doit refléter une décision métier assumée, pas un ordre choisi par hasard entre deux lignes.*

#### 1. ROW\_NUMBER() pour dédupliquer

**Cas :** La table `adresses_clients` garde un historique d'adresses, il faut ne garder que la plus récente par client.

```
SELECT *
FROM (
  SELECT
    client_id,
    adresse,
    maj_le,
    ROW_NUMBER() OVER (
      PARTITION BY client_id
      ORDER BY maj_le DESC
    ) AS rn
  FROM adresses_clients
) t
WHERE rn = 1;
```

**Le piège :** oublier le `ORDER BY` dans le `OVER`, ou partitionner sur la mauvaise colonne. Sans tri explicite, le rang 1 devient arbitraire : la requête tourne, mais elle ne garantit plus « la dernière adresse ».

**La correction :** nommer le tri qui définit « la dernière ligne », avec un départage explicite si deux lignes partagent la même date.

**Le signal :** tu ne considères jamais l'ordre naturel d'une table comme garanti.

#### 2. ROW\_NUMBER vs RANK vs DENSE\_RANK sur les ex aequo PIÈGE CLASSIQUE

**Cas :** Classement des vendeurs par CA du mois, deux vendeurs à égalité. Sur (100, 100, 90) : `ROW_NUMBER` donne 1, 2, 3 ; `RANK` donne 1, 1, 3 ; `DENSE_RANK` donne 1, 1, 2.

```
SELECT
  vendeur,
  ca,
  ROW_NUMBER() OVER (ORDER BY ca DESC) AS rn,
  RANK() OVER (ORDER BY ca DESC) AS rnk,
  DENSE_RANK() OVER (ORDER BY ca DESC) AS drnk
FROM ventes_mensuelles;
```

**Le piège :** utiliser `ROW_NUMBER` pour un « top 3 » quand deux vendeurs sont à égalité en 2e position. Elle tranche arbitrairement lequel entre dans le top, sans que ce soit une décision métier assumée.

**La correction :** `ROW_NUMBER` pour dédupliquer, `RANK` ou `DENSE_RANK` dès que l'égalité doit compter.

**Le signal :** le choix de la fonction est une décision métier, pas un détail de syntaxe interchangeable.

### 3. NTILE(5) pour des quintiles (scoring RFM)

**Cas :** Segmenter les clients en 5 groupes de valeur pour un scoring RFM.

```
SELECT
  client_id,
  montant_total,
  NTILE(5) OVER (ORDER BY montant_total DESC) AS quintile
FROM clients_valeur;
```

**Le piège :** NTILE répartit un nombre égal de lignes par groupe, pas une plage de valeurs égale. Deux clients à montant presque identique peuvent tomber dans des quintiles différents, juste de part et d'autre du seuil de répartition.

**La correction :** vérifier la distribution réelle avant de figer des seuils métier sur les bornes. « Quintile 1 » veut dire les 20% de lignes triées, pas « les clients au dessus de X euros ».

**Le signal :** connaître cette limite évite de vendre un scoring RFM biaisé au métier.

## Cumuler sans se tromper.

*Ici, le cadre de la fenêtre décide si ton cumul avance ligne à ligne ou saute par blocs entiers.*

### 4. Running total : le piège du cadre par défaut PIÈGE CENTRAL

**Cas :** Cumul de CA jour après jour.

```
SELECT
  date_vente,
  ca,
  SUM(ca) OVER (ORDER BY date_vente) AS cumul
FROM ventes_jour;
```

**Le piège :** dès qu'un ORDER BY est présent sans cadre explicite, le cadre par défaut est RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW. RANGE compare les valeurs de tri, pas les lignes physiques : deux lignes à la même date reçoivent le même cumul, celui qui inclut déjà toutes les lignes de ce jour.

**La correction :** remplacer le cadre implicite par ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW, qui compte les lignes physiques, pas les valeurs égales.

**Le signal :** connaître la différence RANGE/ROWS est le vrai marqueur de niveau sur les fonctions fenêtre.

```
SELECT
  date_vente,
  ca,
  SUM(ca) OVER (
    ORDER BY date_vente
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS cumul
FROM ventes_jour;
```

#### RANGE vs ROWS, l'essentiel

Le cadre par défaut n'est pas un détail de syntaxe, c'est ce qui décide si ton cumul est fiable.

##### RANGE (défaut avec ORDER BY)

Regroupe les lignes de même valeur de tri : risque de cumul en bloc sur des doublons.

##### ROWS

Compte les lignes physiques : cumul ligne à ligne fiable, même avec des doublons.

### 5. Moyenne mobile 7 jours PIÈGE CLASSIQUE

**Cas :** Lisser le CA quotidien sur une fenêtre glissante de 7 jours.

```
SELECT
  date_vente,
  ca,
  AVG(ca) OVER (
    ORDER BY date_vente
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
  ) AS moyenne_7j
FROM ventes_jour;
```

**Le piège :** si la table a des jours manquants (rupture, flux absent un jour), `ROWS BETWEEN 6 PRECEDING` compte 7 lignes, pas 7 jours calendaires. La fenêtre glisse alors sur plus de 7 jours réels sans aucune erreur.

**La correction :** générer un calendrier complet (jointure avec une table de dates) avant d'appliquer la fenêtre, pour garantir une ligne par jour.

**Le signal :** tu vérifies que « 7 lignes » égale bien « 7 jours » avant de livrer un indicateur de tendance.

## 6. PARTITION BY qui réinitialise le cumul par groupe

**Cas :** Cumul de CA par magasin, pas un cumul global.

```
SELECT
  magasin_id,
  date_vente,
  ca,
  SUM(ca) OVER (
    PARTITION BY magasin_id
    ORDER BY date_vente
    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS cumul_magasin
FROM ventes_jour;
```

**Le piège :** oublier `PARTITION BY magasin_id`. Le cumul continue alors de magasin en magasin dans l'ordre du tri global, sans aucune erreur visible.

**La correction :** toujours partitionner par l'entité qui doit avoir un cumul indépendant, avant le tri qui définit la progression.

**Le signal :** la version avec et sans `PARTITION BY` compile toutes les deux sans erreur : un piège d'entretien classique et silencieux.

## 7. Part du total, SUM() OVER (PARTITION BY) sans GROUP BY

**Cas :** Afficher, pour chaque produit, sa part en pourcentage du CA total de sa catégorie, sans perdre le détail ligne à ligne.

```
SELECT
  categorie,
  produit,
  ca,
  ca / SUM(ca) OVER (PARTITION BY categorie) AS part_categorie
FROM ventes_produits;
```

**Le piège inverse du cas précédent :** ici il n'y a pas de `ORDER BY` dans le `OVER`. Sans lui, le cadre par défaut couvre toute la partition, pas seulement jusqu'à la ligne courante.

**La correction :** rien à corriger, la requête est juste. Retiens la règle qui la rend juste : le cadre par défaut change selon qu'il y a ou non un `ORDER BY`, c'est l'inverse du cas 3.

**Le signal :** cette requête remplace un `JOIN` sur sous-requête agrégée en une seule passe, plus lisible et plus rapide.

## Décaler et repérer.

*Ici, la partition décide si tu compares une entité à elle-même, ou par erreur à sa voisine.*

### 8. LAG/LEAD pour un delta période sur période

**Cas :** Variation du CA mois sur mois, par pays.

```
SELECT
  pays,
  mois,
  ca,
  ca - LAG(ca) OVER (PARTITION BY pays ORDER BY mois) AS delta_mois_precedent,
  LEAD(ca) OVER (PARTITION BY pays ORDER BY mois) AS ca_mois_suivant
FROM ca_mensuel;
```

**Le piège :** oublier `PARTITION BY pays` quand la table contient plusieurs entités. `LAG` va chercher la ligne précédente toutes entités confondues : janvier du pays A se calcule avec décembre du pays B.

**La correction :** `PARTITION BY` sur l'entité, `ORDER BY` sur la période, dans cet ordre.

**Le signal :** `LAG/LEAD` sans partition ne produit aucune erreur SQL, juste un delta qui mélange deux entités différentes.

### 9. FIRST\_VALUE/LAST\_VALUE : le piège du cadre PIÈGE CENTRAL

**Cas :** Afficher, pour chaque commande, le premier et le dernier statut de son historique.

```
SELECT
  commande_id,
  statut,
  horodatage,
  FIRST_VALUE(statut) OVER (
    PARTITION BY commande_id ORDER BY horodatage
  ) AS premier_statut,
  LAST_VALUE(statut) OVER (
    PARTITION BY commande_id ORDER BY horodatage
  ) AS dernier_statut
FROM historique_commandes;
```

**Le piège :** `LAST_VALUE` hérite du même cadre par défaut que le cas 3, `RANGE ... AND CURRENT ROW`. La fenêtre s'arrête à la ligne courante : « dernier statut » renvoie donc le statut de la ligne elle-même, pas le vrai dernier de la partition.

**La correction :** étendre le cadre avec `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING` pour que la fonction regarde toute la partition.

**Le signal, mon avis tranché :** `LAST_VALUE` est la fonction la plus mal nommée de SQL. Son nom promet « la dernière valeur », mais sans cadre explicite elle rend la valeur courante. Si tu ne dois retenir qu'une seule règle de ce guide, retiens celle-là.

```
LAST_VALUE(statut) OVER (
  PARTITION BY commande_id ORDER BY horodatage
  ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
) AS dernier_statut
```

#### Pourquoi ce piège est éliminatoire

`LAST_VALUE` sans cadre explicite ne renvoie jamais d'erreur, juste un mauvais chiffre.

#### Cadre par défaut

`RANGE UNBOUNDED PRECEDING AND CURRENT ROW`.

#### Conséquence

`LAST_VALUE` regarde seulement jusqu'à la ligne courante, jamais après.

## 10. Numéroté les événements par utilisateur pour un funnel PIÈGE CLASSIQUE

**Cas :** Reconstituer le parcours d'un utilisateur (vue, panier, achat) pour analyser où il décroche.

```
SELECT
  utilisateur_id,
  evenement,
  horodatage,
  ROW_NUMBER() OVER (
    PARTITION BY utilisateur_id
    ORDER BY horodatage
  ) AS etape_numero
FROM evenements_tracking;
```

**Le piège :** des événements dupliqués (un clic tracké deux fois par un bug front) faussent la numérotation. Une étape peut sembler franchie deux fois, et le taux de conversion calculé ensuite peut dépasser 100% entre deux étapes.

**La correction :** dédupliquer en amont, sur `utilisateur_id`, `evenement` et une fenêtre de temps courte, ou sur un identifiant d'événement unique, avant de numéroté les étapes.

**Le signal :** un funnel avec un taux de conversion supérieur à 100% entre deux étapes est presque toujours un problème de déduplication, jamais un miracle métier.

## Filtrer et classer par groupe.

*Ici, le moteur SQL et le choix de fonction changent la promesse que tu tiens face au métier.*

## 11. QUALIFY pour filtrer le résultat d'une fonction fenêtre PIÈGE DE DIALECTE

**Cas :** Garder uniquement la dernière commande de chaque client, sans sous-requête.

```
SELECT
  client_id,
  commande_id,
  date_commande
FROM commandes
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY client_id
  ORDER BY date_commande DESC
) = 1;
```

**Le piège :** `QUALIFY` n'existe pas partout. Disponible en Snowflake, BigQuery et Databricks, absent de PostgreSQL, MySQL et SQL Server. L'utiliser sur le mauvais moteur fait planter la requête.

**La correction (dialecte standard) :** passer par une CTE avec la fonction fenêtre, puis un `WHERE` sur la colonne calculée.

**Le signal :** savoir dire « ça dépend du moteur » sur `QUALIFY` est un signe de maturité SQL, pas une réponse évasive.

```
WITH classees AS (
  SELECT
    client_id, commande_id, date_commande,
    ROW_NUMBER() OVER (
      PARTITION BY client_id ORDER BY date_commande DESC
    ) AS rn
  FROM commandes
)
SELECT client_id, commande_id, date_commande
FROM classees
WHERE rn = 1;
```

### Support QUALIFY par moteur

Un réflexe qui rassure en entretien : nommer le moteur avant d'écrire QUALIFY.

#### Disponible

Snowflake, BigQuery, Databricks (Spark SQL).

#### Absent

PostgreSQL, MySQL, SQL Server : passer par une CTE.

## 12. DENSE\_RANK pour un top N par catégorie

**Cas :** Top 3 des produits les plus vendus dans chaque catégorie.

```
WITH classees AS (  
  SELECT  
    categorie,  
    produit,  
    unites_vendues,  
    DENSE_RANK() OVER (  
      PARTITION BY categorie  
      ORDER BY unites_vendues DESC  
    ) AS rang  
  FROM ventes_produits  
)  
SELECT categorie, produit, unites_vendues, rang  
FROM classees  
WHERE rang <= 3;
```

**Le piège :** utiliser ROW\_NUMBER ici tronquerait arbitrairement un ex aequo au rang 3. DENSE\_RANK garde tous les produits à égalité au même rang : le « top 3 » peut renvoyer plus de 3 lignes par catégorie.

**La correction :** choisir la fonction selon la promesse métier. « Top 3 lignes exactement » appelle ROW\_NUMBER. « Top 3 rangs, ex aequo inclus » appelle DENSE\_RANK.

**Le signal :** cette question revient souvent en entretien pour vérifier que tu ne confonds pas « 3 lignes » et « 3 rangs ».

12 sur 12. Le fil commun de ce guide : une fonction fenêtre ne casse jamais la requête quand le cadre ou la partition sont faux, elle change juste le chiffre en silence. Le vrai niveau, ce n'est pas de connaître le nom des fonctions, c'est de vérifier trois choses à chaque fois : le tri qui donne son sens au rang, la partition qui isole les groupes, et le cadre qui délimite ce que la fonction regarde vraiment.

### Fait main, en solo.

Si ce guide t'a aidé, partage-le autour de toi : il peut débloquent un autre candidat. Et si tu veux que je traite un sujet précis, dis-le moi en commentaire : c'est là que je pioche mes prochains guides.

**Dylan**

